

Staking Rewards *Billions*

HALBORN

Staking Rewards - Billions

Prepared by:  HALBORN

Last Updated 03/23/2026

Date of Engagement: March 19th, 2026 - March 19th, 2026

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
4	0	0	0	2	2

TABLE OF CONTENTS

1. Introduction	
2. Assessment summary	
3. Test approach & methodology	
4. Risk methodology	
5. Scope	
6. Assessment summary & findings overview	
7. Findings & Tech Details	
7.1 No upper bound on setinitiallockperiod duration allows permanent fund locking	
7.2 Exit() reverts when user's full balance is locked	
7.3 Third-party default_admin_role holders persist through ownership transfers	
7.4 Renounceownership() permanently destroys accesscontrol role management	

1. INTRODUCTION

Billions engaged Halborn to perform a security assessment of their smart contracts on March 19th, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The **StakingRewards** contract is an upgradeable Synthetix-fork staking contract where users stake BILL tokens to earn BILL rewards. It adds per-user time-locked staking, delegated on-behalf operations, and a global launch lock period on top of the standard reward distribution model, governed by a timelock and multisig.

2. ASSESSMENT SUMMARY

Halborn was allocated 1 day for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, **Halborn** identified several areas for improvement to reduce the likelihood and impact of security risks, which were successfully addressed by the **Billions team**. The main recommendations were:

- `Skip withdraw() when unlocked amount is zero instead of reverting.`
- `Cap the duration parameter to a reasonable maximum.`
- `Override the renounceOwnership() function to revert.`
- `Override grantRole() to restrict DEFAULT_ADMIN_ROLE to the current owner only.`

3. TEST APPROACH & METHODOLOGY

Halborn conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.

The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts.
- Fork testing against main networks.
- Static security analysis of scoped contracts, and imported functions (**Slither**).

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

Impact Metric (M_I)	Metric Value	Numerical Value
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (<i>r</i>)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (<i>s</i>)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

5. SCOPE

REPOSITORY
(a) Repository: <code>billions-token</code> (b) Assessed Commit ID: <code>4b7e38f</code> (c) Items in scope: <code>contracts/staking/StakingRewards.sol</code>

FILE
(a) Submitted File: <code>contracts.zip</code>

REMEDIATION COMMIT ID:
<code>5250377</code>
<code>Out-of-Scope</code>: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	0	0	2	2

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
NO UPPER BOUND ON SETINITIALLOCKPERIOD DURATION ALLOWS PERMANENT FUND LOCKING	LOW	SOLVED - 03/18/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
EXIT() REVERTS WHEN USER'S FULL BALANCE IS LOCKED	LOW	SOLVED - 03/18/2026
THIRD-PARTY DEFAULT_ADMIN_ROLE HOLDERS PERSIST THROUGH OWNERSHIP TRANSFERS	INFORMATIONAL	SOLVED - 03/18/2026
RENOUNCEOWNERSHIP() PERMANENTLY DESTROYS ACCESSCONTROL ROLE MANAGEMENT	INFORMATIONAL	SOLVED - 03/18/2026

7. FINDINGS & TECH DETAILS

7.1 NO UPPER BOUND ON SETINITIALLOCKPERIOD DURATION ALLOWS PERMANENT FUND LOCKING

// LOW

Description

The `setInitialLockPeriod()` function accepts an unbounded `duration` parameter with no maximum cap. An owner can set `initialLockPeriodFinish` to a timestamp that `block.timestamp` may never reach, permanently preventing all users from calling `withdraw()` or `getReward()`.

The `setInitialLockPeriod()` function at line 392 validates that `duration` is greater than zero and that the previous lock period has completed, but does not enforce an upper bound on the `duration` value. The resulting `initialLockPeriodFinish` is computed as `block.timestamp + duration`. Both `withdraw()` and `getReward()` require `block.timestamp >= initialLockPeriodFinish` before allowing execution. If `duration` is mistakenly set to an arbitrarily high value, the resulting `initialLockPeriodFinish` may never be feasibly reached. Since `exit()` delegates to `withdraw()` internally, it is also affected. There's a logic guard that prevents setting a new lock period while a current one is active, but this does not limit the magnitude of the initial or subsequent lock. On the first call after deployment, `initialLockPeriodFinish` defaults to zero, so the guard passes unconditionally.

In the event of this scenario, all staked tokens and all accrued rewards become permanently inaccessible. Users cannot withdraw their principal via `withdraw()`, cannot claim earned rewards via `getReward()`, and cannot use the convenience `exit()` function. The `recoverERC20()` function allows the owner to recover non-staked token balances, but individual users have no recovery path for their staked positions. The effect is equivalent to a permanent freeze of all user funds held by the contract.

Code Location

StakingRewards.sol:

 Copy Code

```
1 function setInitialLockPeriod(uint256 duration) external onlyOwner {
2     require(duration > 0, "Initial lock period must be greater than 0");
3     require(
4         block.timestamp >= initialLockPeriodFinish,
5         "Previous initial lock period must be complete before changing the duration for the new period"
6     );
7     initialLockPeriodDuration = duration;
8     initialLockPeriodFinish = block.timestamp + duration; // ← no upper bound on duration
9     emit InitialLockPeriodUpdated(duration);
10 }
```

StakingRewards.sol:

```
310 | require(  
311 |     block.timestamp >= initialLockPeriodFinish, // ← blocks all exits when finish is unreachable  
312 |     "Withdraw not allowed during initial lock period"  
313 | );
```

BVSS

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:N/D:N/Y:H (2.5)

Recommendation

Consider adding a maximum duration constant to bound the lock period to a reasonable operational range.

Remediation Comment

SOLVED: The **Billions team** solved this finding in the specified commit by introducing a `MAX_INITIAL_LOCK_PERIOD_DURATION` constant of 730 days and adding a require check in `setInitialLockPeriod()` that rejects any duration exceeding this cap.

Remediation Hash

52503776e7a6131c696fbcc739fcbc484107dbea

7.2 EXIT() REVERTS WHEN USER'S FULL BALANCE IS LOCKED

// LOW

Description

The `exit()` function unconditionally calls `withdraw()` before calling `getReward()`. When a user's entire staked balance is locked, the computed withdrawal amount is zero, causing `withdraw()` to revert and preventing the user from claiming accrued rewards through `exit()`. This contradicts the documented behavior in the contract's changelog.

The `exit()` function computes the withdrawal amount as `_balances[msg.sender] - getLockedStakeAmount(msg.sender)` and passes it directly to `withdraw()`. When a user has locked all of their staked tokens, this expression evaluates to zero. The `withdraw()` function contains `require(amount > 0, "Cannot withdraw 0")`, which causes the entire `exit()` call to revert. Because `getReward()` is called after `withdraw()`, the reward claim is never reached. The contract's changelog states that `exit()` was "Modified exit() to withdraw only unlocked tokens (skips withdraw if all tokens are locked)," indicating the intended behavior is to skip the withdrawal gracefully rather than revert. Users in this state can still call `getReward()` directly as a standalone transaction, so accrued rewards are not permanently lost.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

Consider adding a conditional check to skip the withdrawal when the unlocked amount is zero, aligning the implementation with the documented behavior.

Remediation Comment

SOLVED: The **Billions team** solved this finding in the specified commit by wrapping the `withdraw()` call inside `exit()` with a conditional check that skips it when the unlocked amount is zero, allowing `getReward()` to execute regardless of lock status.

Remediation Hash

52503776e7a6131c696fbcc739fcbc484107dbea

7.3 THIRD-PARTY `DEFAULT\_ADMIN\_ROLE` HOLDERS PERSIST THROUGH OWNERSHIP TRANSFERS

// INFORMATIONAL

Description

The `StakingRewards` contract's `_transferOwnership()` override synchronizes `DEFAULT_ADMIN_ROLE` only between the outgoing and incoming owner, leaving any additional `DEFAULT_ADMIN_ROLE` holders untouched. This may result in undisclosed administrative access persisting after an ownership transfer, with the potential for unauthorized role management.

`StakingRewards` inherits both `Ownable2StepUpgradeable` and `AccessControlUpgradeable`, linking the two systems through a `_transferOwnership()` override. During initialization, `DEFAULT_ADMIN_ROLE` is granted to the owner. The override in the `_transferOwnership` function grants `DEFAULT_ADMIN_ROLE` to the new owner and revokes it from the old owner, but it does not account for other addresses that may have been granted `DEFAULT_ADMIN_ROLE` independently via `grantRole()`. Since `DEFAULT_ADMIN_ROLE` is its own admin role in OpenZeppelin's `AccessControlUpgradeable`, any holder of this role can grant it to additional addresses at any time. These additional holders are not tracked by the ownership transfer logic and retain full role-management capability after the transfer completes. The contract uses `AccessControlUpgradeable` rather than `AccessControlEnumerableUpgradeable`, so there is no on-chain mechanism for the new owner to enumerate existing role holders and discover the residual access.

An address retaining `DEFAULT_ADMIN_ROLE` after ownership transfer can independently grant `REWARDS_DISTRIBUTOR_ROLE` to arbitrary addresses, enabling manipulation of reward distribution via `notifyRewardAmount()`. It can also grant `STAKER_ON_BEHALF_ROLE`, enabling the lock escalation behavior. These actions affect protocol funds and user positions without the current owner's knowledge or consent.

Code Location

`StakingRewards.sol`:

 Copy Code

```
517 function _transferOwnership(address newOwner) internal override {
518     address oldOwner = owner();
519     super._transferOwnership(newOwner);
520     // Ensure the new owner has the necessary roles
521     if (newOwner != address(0) && !hasRole(DEFAULT_ADMIN_ROLE, newOwner)) {
522         _grantRole(DEFAULT_ADMIN_ROLE, newOwner); // ← grants to new owner
523     }
524     if (oldOwner != newOwner) {
525         // Revoke roles from the old owner
526         _revokeRole(DEFAULT_ADMIN_ROLE, oldOwner); // ← revokes from old owner only
527     }
528 }
```

`StakingRewards.sol`:

```
144 | _grantRole(DEFAULT_ADMIN_ROLE, _owner);
```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (1.0)

Recommendation

Consider restricting `DEFAULT_ADMIN_ROLE` so that it can only be held by the current `owner()`. Alternatively, consider migrating from `AccessControlUpgradeable` to `AccessControlDefaultAdminRulesUpgradeable`, which enforces a time-delayed, single-holder model for the admin role that aligns with the two-step safety pattern already used for ownership transfers.

Remediation Comment

SOLVED: The **Billions team** solved this finding in the specified commit by replacing the dual `Ownable2StepUpgradeable + AccessControlUpgradeable` architecture with OpenZeppelin's `AccessControlDefaultAdminRulesUpgradeable`, which structurally enforces a single `DEFAULT_ADMIN_ROLE` holder and blocks `grantRole()` for the admin role entirely, eliminating third-party admin persistence by design.

Remediation Hash

52503776e7a6131c696fbcc739fcbc484107dbea

7.4 RENOUNCEOWNERSHIP() PERMANENTLY DESTROYS ACCESSCONTROL ROLE MANAGEMENT

// INFORMATIONAL

Description

Calling `renounceOwnership()` triggers the `_transferOwnership()` override with `address(0)` as the new owner. The override revokes `DEFAULT_ADMIN_ROLE` from the outgoing owner without granting it to any replacement address, permanently removing the ability to manage `REWARDS_DISTRIBUTOR_ROLE` and `STAKER_ON_BEHALF_ROLE`.

The `_transferOwnership()` override handles ownership changes by granting `DEFAULT_ADMIN_ROLE` to the new owner and revoking it from the old owner. When `renounceOwnership()` is called, it invokes `_transferOwnership(address(0))`. The condition `(newOwner != address(0))` evaluates to false, so `DEFAULT_ADMIN_ROLE` is not granted to the zero address. The condition `(oldOwner != newOwner)` evaluates to true (assuming the old owner is non-zero), so `DEFAULT_ADMIN_ROLE` is revoked from the old owner. If no other address holds `DEFAULT_ADMIN_ROLE`, the role has no remaining holders. Since `DEFAULT_ADMIN_ROLE` is the admin role for both `REWARDS_DISTRIBUTOR_ROLE` and `STAKER_ON_BEHALF_ROLE` by default in OpenZeppelin's `AccessControlUpgradeable`, no address can subsequently call `grantRole()` or `revokeRole()` for any role. The current role holders become permanently fixed, they cannot be replaced if compromised, and no new holders can be added.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (1.0)

Recommendation

Consider overriding `renounceOwnership()` to revert, preventing the cascading loss of role management. Alternatively, consider documenting the cascading effect prominently in the function's NatSpec so that any future caller is aware that renouncing ownership also permanently freezes the AccessControl role system.

Remediation Comment

SOLVED: The **Billions team** solved this finding in the specified commit by replacing `Ownable2StepUpgradeable` with `AccessControlDefaultAdminRulesUpgradeable`, which removes the `renounceOwnership()` function entirely and requires a deliberate 2-step process with a mandatory time delay to renounce the admin role, preventing accidental destruction of role management.

Remediation Hash

52503776e7a6131c696fbcc739fcbc484107dbea

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.